Card Fraud Triage via Gaussian Mixture Density Scoring and Autoencoder Anomaly Detection

Yash Ram Archit Phalak Preet
Student project team

ABSTRACT

Abstract—We compare two unsupervised anomaly detectors for credit-card fraud: a Gaussian Mixture Model (GMM) scored by negative log-likelihood and an undercomplete autoencoder scored by reconstruction error. Both are trained only on legitimate transactions from the public ULB dataset (284,807 transactions, 0.173% fraud), tuned and thresholded on a validation set with a 1% false-positive budget, and evaluated once on the same untouched test set of 56,746 transactions (95 frauds). The GMM (six components, chosen by BIC) achieved test average precision 0.473 and 49 frauds in its top 100 alerts; the selected autoencoder (bottleneck 8) achieved 0.358 and 41. Random ranking gives 0.0017. At the validation-chosen thresholds all models caught the same 74 of 95 frauds with about 11% precision, and the GMM and autoencoder missed 20 of the same frauds. The comparison is limited by only 95 test frauds, one random two-day split, one seed and anonymised PCA features.

Index Terms—credit-card fraud, anomaly detection, Gaussian mixture model, autoencoder, average precision, class imbalance

I. Introduction and Problem Statement

Card fraud is rare: in our dataset only 0.173% of transactions are fraudulent. A classifier that never flags anything is therefore 99.83% accurate and useless. Instead of training a supervised classifier on so few fraud labels, we teach two models what a legitimate transaction looks like and rank transactions by how unusual they are, so that analysts can review the most suspicious ones first. This is a *triage* task: the output is a ranked list and an alert threshold, not a verdict.

The question is whether a density model (GMM) or a neural network (autoencoder) ranks fraud better when both see the same 29 features, the same training rows and the same evaluation protocol. The outcome is open: we report what we measure. Both models are trained only on legitimate transactions, thresholds are fixed on validation data, and the test set is used once.

II. Literature Review and Related Work

A. Dataset

The Credit Card Fraud Detection dataset was released by the ULB Machine Learning Group in collaboration with Worldline and has been used in published work on fraud detection with imbalanced classes [1]. It contains transactions made by European cardholders over two days in September 2013. All inputs except `Time` and `Amount` are anonymised principal components, so no feature can be interpreted in business terms.

B. Density and Reconstruction Based Detection

A Gaussian mixture models a density as a weighted sum of Gaussians fitted by expectation–maximisation [2, 3]; the number of components can be compared with information criteria such as BIC [4]. An autoencoder learns to compress and rebuild its input through a narrow bottleneck [5]; inputs that are rebuilt badly are treated as anomalies. Both ideas are standard for one-class or novelty detection.

C. Evaluation under Imbalance

For rare positives, precision–recall analysis is more informative than ROC analysis or accuracy [6, 7]. We therefore use average precision, precision at a fixed number of alerts and recall at a false-positive budget.



Figure 1. Class distribution of the raw dataset on a log scale.

D. Relation to This Work

Neither the dataset paper nor the methodological references above ran our experiment, so we draw no conclusion from their results; our findings are the measurements in Section 8.

III. Dataset Description and EDA

A. Dataset

The file has 284,807 rows and 31 columns: `Time`, `V1`–`V28`, `Amount` and `Class`. There are no missing values. Of the rows, 492 (0.173%) are fraud and 284,315 are legitimate (Fig. 1). `Time` spans about 48 hours. `Amount` is strongly right-skewed (mean 88.35, median 22.00, maximum 25,691.16), and 1,825 transactions have amount zero.

B. Data Quality

We found 1,081 exact duplicate rows. Removing them leaves 283,726 rows, of which 473 are fraud. Only exact copies were checked; near-duplicates were not examined.

C. The Accuracy Trap

Predicting "legitimate" for every transaction gives accuracy 99.8273% while catching no fraud. Accuracy is therefore not reported as a result. The random-ranking baseline for average precision equals the fraud rate of the test set, $95/56{,}746 = 0.0017$.

IV. Methodology and Preprocessing

A. Features

The model inputs are the 28 PCA components `V1`–`V28` and $\texttt{Amount_log} = \log(1 + \texttt{Amount})$, giving $d = 29$ features. The

Table 1
DATA SPLITS USED IN ALL EXPERIMENTS.

Split	Rows	Frauds	Use
fit	152,955	0	scaler, GMM, AE weights
es	16,996	0	AE early stopping
val	56,745	94	AE choice, thresholds
test	56,746	95	final evaluation, once

log transform compresses large amounts (25,691 maps to about 10.2) and is defined for zero amounts. `Time` is excluded because it only counts seconds from the start of the recording and says nothing about the transaction itself. `Class` is never a model input; this is asserted in code.

B. Splitting

Duplicates are removed before splitting. The data are split with stratification into 60% training, 20% validation and 20% test (Table 1). Fraud rows in the training part (284) are discarded, so the models never see a fraud example. The legitimate training rows are divided again: 90% for fitting and 10% held out for autoencoder early stopping. The split is random, not chronological.

C. Scaling and Leakage Control

A `StandardScaler` is fitted on the 152,955 legitimate fitting rows only and applied to every split. Duplicates are removed before splitting, the GMM and autoencoder are fitted only on legitimate training rows, thresholds and the autoencoder configuration use validation data only, and the test set is touched once at the end.

V. GAUSSIAN MIXTURE MODEL

A. Model and Score

A GMM with K components models $p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k)$, with weights π_k summing to one and a full 29×29 covariance per component. It is fitted by EM on legitimate rows (scikit-learn [9], `covariance_type="full"`, seed 42). The anomaly score is the negative log-likelihood, $s_{\mathrm{GMM}}(\mathbf{x}) = -\log p(\mathbf{x})$; higher means more suspicious. Because p is a density, scores can be negative (37.8% of validation rows), and only the ranking matters.

B. Choosing K and Regularisation

We fitted $K \in \{1,2,3,4,6,8\}$ on a random 50,000-row subsample of the fitting rows and compared AIC and BIC (Table 2, Fig. 2). Both are lowest at $K = 6$ (BIC 705,261), so $K = 6$ was selected and refitted on all 152,955 rows; EM converged in 27 iterations with 2,789 parameters ($6 \cdot 29 + 6 \cdot 435 + 5$). With the default covariance regularisation (`reg_covar`$= 10^{-6}$) some components collapsed onto near-singular directions and BIC never reached a minimum. We therefore set `reg_covar`$= 10^{-3}$, a decision based on behaviour on training data before any test result was seen.

VI. AUTOENCODER

A. Architecture and Training

The autoencoder is a PyTorch [10] network $29 \to 16 \to b \to 16 \to 29$ with ReLU in the hidden layers and a linear output, since standardised inputs can be negative. For bottleneck $b = 8$ it has 1,253 parameters. It is trained on the fitting rows with

Table 2
GMM MODEL SELECTION ON LEGITIMATE TRAINING ROWS.

K	AIC	BIC
1	4,032,850	4,036,943
2	2,228,812	2,237,006
3	2,139,656	2,151,950
4	1,651,352	1,667,748
6	**680,663**	**705,261**
8	789,664	822,465

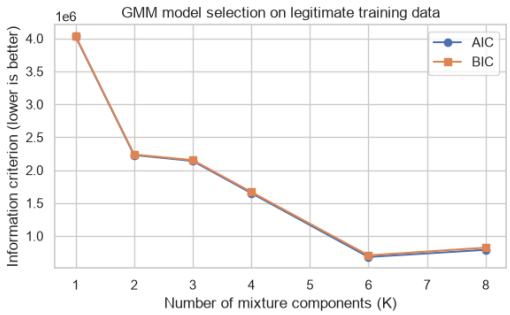


Figure 2. AIC and BIC against the number of components. Both are lowest at $K = 6$.

Table 3
AUTOENCODER CONFIGURATIONS (VALIDATION).

Config	b	Epochs/pat.	Best MSE (ep.)	Stopped	Val AP
AE-base	8	50/5	0.4353 (49)	cap	0.405
AE-b8	8	100/8	0.4256 (99)	cap	**0.413**
AE-b4	4	100/8	0.5055 (100)	cap	0.377
AE-b12	12	100/8	0.4023 (65)	early (73)	0.282

mean squared error, Adam [8] (learning rate 10^{-3}), batch size 512 (299 batches per epoch) and seed 42, with early stopping on the held-out legitimate rows. The anomaly score is the per-transaction reconstruction error, $s_{\mathrm{AE}}(\mathbf{x}) = \frac{1}{29}\sum_j (x_j - \hat{x}_j)^2$; higher means more suspicious.

B. Experiments

The baseline (bottleneck 8, at most 50 epochs, patience 5) was still improving when it reached its epoch cap. We therefore ran Experiment A (100 epochs, patience 8) and Experiment B (bottlenecks 4, 8 and 12, each with 100 epochs and patience 8), keeping data, features, hidden size and seed unchanged. The configuration was selected by validation average precision (Table 3, Fig. 3). AE-b8 was selected; it again stopped at its epoch cap, so it is not fully converged. The widest bottleneck rebuilt legitimate data best (lowest loss) but ranked fraud worst, so a lower reconstruction loss did not mean better detection.

VII. THRESHOLDS AND EVALUATION PROTOCOL

A. Alert Thresholds

For each model the alert threshold is the 99th percentile of that model's scores on legitimate validation transactions, a false-positive budget of about 1%. A transaction is flagged when its score exceeds the threshold. Thresholds were 57.5098 (GMM), 1.9348 (AE baseline) and 1.8553 (AE selected). Each threshold lets through exactly 567 of 56,651 legitimate validation rows (1.0009%). On validation the GMM caught 81

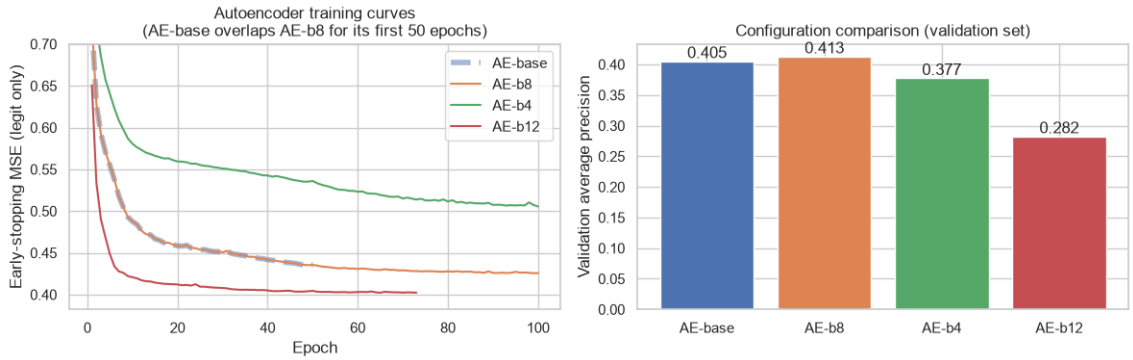


Figure 3. Autoencoder experiments. Left: early-stopping MSE per epoch for each configuration (AE-base overlaps AE-b8 for its first 50 epochs). Right: validation average precision by configuration.

Table 4
TEST PERFORMANCE (56,746 ROWS, 95 FRAUDS). THRESHOLDS FIXED ON VALIDATION DATA.

Metric	GMM	AE base	AE b8
Average precision	**0.473**	0.357	0.358
Precision@100	**0.49**	0.41	0.41
Recall @ 1% FPR (test ROC)	0.779	0.779	0.779
Recall @ val. threshold	0.779	0.779	0.779
TP / FP / FN	74/620/21	74/589/21	74/593/21
Flagged	694	663	667
Precision @ val. thr.	0.107	0.112	0.111
Test FPR @ val. thr.	1.09%	1.04%	1.05%

of 94 frauds, the baseline autoencoder 79 and the selected autoencoder 80.

B. Metrics

Average precision (AP) summarises the precision–recall curve and has the fraud rate as its random baseline. Precision@100 is the fraction of fraud among the 100 highest-scoring transactions. We report two different recall numbers. *Recall at 1% FPR (test ROC)* uses a cut-off derived from the test labels: it measures ranking quality and is not achievable in practice. *Recall at the validation threshold* uses the threshold fixed on validation data and is what a deployed system would obtain. Confusion matrices have the true class in rows and the predicted class in columns.

VIII. Results and Comparative Analysis

Table 4 gives test results. The GMM had the highest average precision (0.473 against 0.357 and 0.358) and 49 frauds in its top 100 alerts against 41; all three models are far above random ranking (0.0017). The precision–recall curves (Fig. 6) show the GMM above the autoencoders over most recall levels, and the curves converge near the operating point. At a 1% false-positive rate all models reach recall 0.779, and at the validation-chosen thresholds all three catch the same 74 of 95 frauds while flagging 694, 663 and 667 transactions (precision about 0.11, test FPR 1.04–1.09%). The two recall values coincide by chance and are different measurements.

The tuning of the autoencoder did not carry over to the test set: AE-b8 beat the baseline on validation AP (0.413 against 0.405), but test AP was 0.3575 against 0.3568, and every other metric is the same or differs by a few flagged transactions.

IX. Operating Trade-offs and Missed Frauds

Fig. 5 shows recall against false-positive rate and the precision and recall obtained when reviewing the top K alerts. Reviewing about 700 alerts yields roughly one fraud per nine alerts at the operating point.

We checked the missed frauds one by one. The GMM and the selected autoencoder each miss 21 test frauds, and 20 of them are the same transactions; each model catches exactly one fraud that the other misses. The baseline and selected autoencoders miss exactly the same 21. About 20% of test frauds therefore look completely normal to both detectors in these features.

X. Discussion, Limitations and Future Improvements

The simpler GMM ranked fraud best, and tuning the autoencoder did not close the gap. The GMM has a clear probabilistic meaning, few settings and an objective size choice, but it assumes Gaussian blobs, has many parameters in 29 dimensions and needed covariance regularisation. The autoencoder can learn non-linear structure but has more settings, and its results depend strongly on the bottleneck (validation AP 0.28–0.41).

Several limitations qualify these findings.

- Only 95 test frauds exist, so one fraud moves recall by about one point; there is one seed, one split and no confidence intervals.
- The split is random over two days, so results do not show performance on future transactions.
- V1–V28 are PCA components fitted by the data provider on the whole dataset, and they cannot be interpreted.
- Validation data were used both to choose the autoencoder and to set thresholds, so the selected validation AP is slightly optimistic.
- The models are trained on legitimate rows only; labels are used to filter training rows, choose the configuration, set thresholds and evaluate. This is one-class detection, not fully label-free.
- Only six GMM sizes and three bottlenecks were tried, AIC/BIC used a 50,000-row subsample, and AE-b8 stopped at its epoch cap.
- Scores are rankings, not probabilities.

Future work should use repeated splits or bootstrapping for confidence intervals, a chronological split with an hour-of-day feature, other covariance types and more initialisations, more autoencoder tuning or a variational autoencoder, Isolation Forest and One-Class SVM baselines, a combination of the

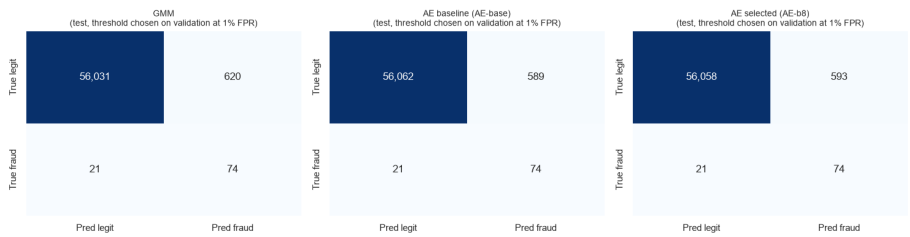


Figure 4. Test confusion matrices at the validation-chosen thresholds (rows: truth, columns: prediction). Left to right: GMM, AE baseline, AE selected.

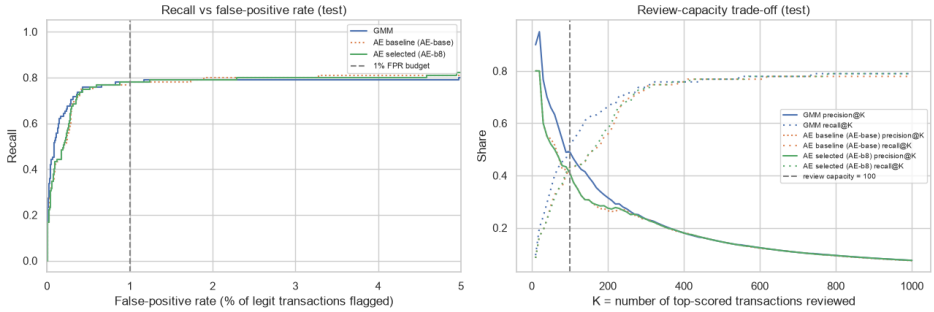


Figure 5. Left: recall against false-positive rate (0–5%). Right: precision and recall when reviewing the top-K scored transactions; the dashed line marks a review capacity of 100.

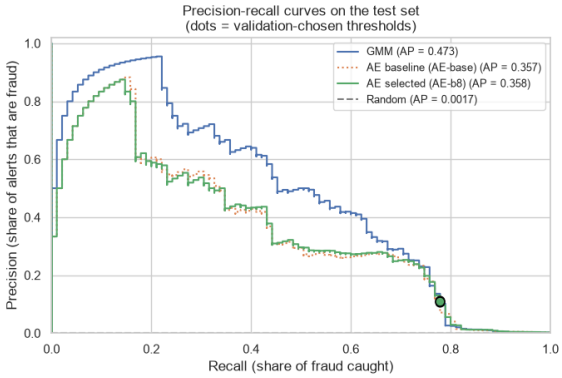


Figure 6. Precision–recall curves on the test set; dots mark the validation-chosen thresholds.

two scores, and a model that uses the fraud labels.

XI. Conclusion

For this benchmark, two unsupervised detectors trained only on legitimate transactions ranked fraud far above chance (AP 0.47 and 0.36 against 0.0017). At a 1% false-positive budget fixed on validation data, both caught 74 of 95 test frauds with about 11% precision. The simpler GMM was at least as good as the autoencoder and ranked fraud best, while the autoencoder experiments did not meaningfully improve test performance. With only 95 test frauds and a single split, the differences are indicative rather than conclusive, and the results do not yet support deployment without evaluation on future data.

Acknowledgment

The repository snapshot did not specify external funding or additional acknowledgments.

References

[1] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in *Proc. IEEE Symp. Series on Computational Intelligence (SSCI)*, 2015.

[2] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the EM algorithm," *J. Royal Statistical Society B*, vol. 39, no. 1, pp. 1–38, 1977.

[3] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

[4] G. Schwarz, "Estimating the dimension of a model," *The Annals of Statistics*, vol. 6, no. 2, pp. 461–464, 1978.

[5] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, 2006.

[6] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proc. ICML*, 2006.

[7] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.

[8] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.

[9] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[10] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, 2019.